ASRAM
LIVE DEMONSTRATION RUNBOOK

Repeatable startup, live ESP32 telemetry, AI demo checks, and safe shutdown

Verified local configuration | 1 November 2026

Purpose

Use this runbook whenever you need to start the ASRAM local demonstration from a fresh Windows session. It records the working arrangement established during the 1 November 2026 troubleshooting session: the updated Python API Gateway on port 8080, ESP32 USB serial ingestion on COM5, and the local Streamlit dashboard on port 8502.

Verified project configuration

Item	Working value
Project folder	C:\Downloadss\aviation_safety_system_milestone13\aviation_safety_system_milestone13
Python environment	.venv (activate with .\.venv\Scripts\Activate.ps1)
ESP32 USB serial	COM5 - Silicon Labs CP210x USB to UART Bridge
Serial speed	115200 baud; newline-delimited JSON telemetry
Local API	http://127.0.0.1:8080
Streamlit dashboard	http://127.0.0.1:8502
SQLite database	Project database; do not delete or reset it

What a successful run looked like

30	4	0/100	0.3 s
Telemetry samples	Logged events	Risk score / LOW	Latest packet age

The final verified screenshot also showed data source API:SERIAL:ESP32 and DHT11, BMP280, MPU6050, and IR modules reporting OK. Counts and risk values will naturally change during later demonstrations.

Important architecture rule
Use the updated local Python API for the ESP32_ENV single-node packets. The older Docker API container expected SYSTEM_AGGREGATE packets and returned HTTP 422. Do not run both API gateways on port 8080.

1. Preflight and start the API Gateway

Before starting services

- Power the ESP32 and connect it to the computer using its USB data cable.
- Close Arduino IDE Serial Monitor. Only one application can normally hold COM5 at a time.
- Confirm Windows sees the CP210x USB-to-UART device. The working port in the verified session was COM5.
- Open PowerShell. Use the same project folder in every terminal.

Check serial ports (run if COM5 is missing)

```
[System.IO.Ports.SerialPort]::GetPortNames()
Get-CimInstance Win32_SerialPort |
Format-Table DeviceID, Name -AutoSize
```

Avoid the old Docker API binding on port 8080

The machine previously had a Docker container named aviation_safety_system_v10-ingest_api-1 publishing 127.0.0.1:8080. That older gateway rejected the ESP32_ENV packet contract. Before starting the local Python API, check whether that container is running:

Check Docker containers and stop only the old API container if running

```
docker ps --format "table {{.Names}}\t{{.Ports}}\t{{.Status}}"
$running = docker ps --format "{{.Names}}"
if ($running -contains "aviation_safety_system_v10-ingest_api-1") {
docker stop aviation_safety_system_v10-ingest_api-1
}
```

Do not remove containers or volumes. Do not run docker compose down -v. The older dashboard containers do not need to be stopped for this workflow.

Terminal 1 - start the updated local API

Open Terminal 1 and run the following block. It reads the existing token from the project .env file without printing it, then starts the project API on 127.0.0.1:8080.

Terminal 1 - API startup (keep this terminal open)

```
cd "C:\Downloadss\aviation_safety_system_milestone13\aviation_safety_system_milestone13"
.\.venv\Scripts\Activate.ps1

$tokenLine = Get-Content .env | Where-Object {
$_ -match '^\s*AVIATION_INGEST_TOKEN\s*='
} | Select-Object -Last 1
if (!$tokenLine) { throw "AVIATION_INGEST_TOKEN not found in .env" }
$token = ($tokenLine -replace '^\s*AVIATION_INGEST_TOKEN\s*=\s*', '').Trim().Trim('"').Trim("'")
if ([string]::IsNullOrWhiteSpace($token)) { throw "Token is empty" }

$env:AVIATION_INGEST_TOKEN = $token
$env:AVIATION_INGEST_HOST = "127.0.0.1"
$env:AVIATION_INGEST_PORT = "8080"
python ingest_api.py
```

Keep Terminal 1 running. Do not launch another API process on port 8080.

Verify API health in a second PowerShell window

Health check

```
Invoke-RestMethod http://127.0.0.1:8080/health
```

Expected health response: ok = True, service = aviation-telemetry-gateway, database = ready. If health fails, fix the API before starting serial ingestion.

2. Start ESP32 serial ingestion

Terminal 2 - connect COM5 and forward telemetry

Open a second PowerShell window. Close Arduino Serial Monitor before running this command. Load the same token from the same project .env file; the local API and the serial bridge must use matching tokens.

Terminal 2 - serial ingestion startup

```
cd "C:\Downloadss\aviation_safety_system_milestone13\aviation_safety_system_milestone13"
.\.venv\Scripts\Activate.ps1

$tokenLine = Get-Content .env | Where-Object {
$_ -match '^\s*AVIATION_INGEST_TOKEN\s*='
} | Select-Object -Last 1
if (!$tokenLine) { throw "AVIATION_INGEST_TOKEN not found in .env" }
$token = ($tokenLine -replace '^\s*AVIATION_INGEST_TOKEN\s*=\s*', '').Trim().Trim('"').Trim("'")
if ([string]::IsNullOrWhiteSpace($token)) { throw "Token is empty" }

$env:AVIATION_INGEST_TOKEN = $token
$env:AVIATION_INGEST_HOST = "127.0.0.1"
python tools_esp32_serial_ingest.py --port COM5 --baud 115200
```

What to expect

- The bridge prints that it is listening to COM5 at 115200 and forwarding to http://127.0.0.1:8080.
- Opening the serial connection can reset the ESP32. Initial startup text, blank lines, or non-JSON lines may be skipped.
- After startup, valid newline-delimited ESP32_ENV JSON packets should be accepted by the API.
- Keep Terminal 2 open during the demonstration. It owns COM5 while running.

Healthy serial path
The previous HTTP 401 problem was resolved by loading the same token used by the running API. The later HTTP 422 was caused by the older Docker API contract; stop that old API container and use the local Python API described on page 2.

If Windows assigns a different COM number

Run the port-discovery commands on page 2. Use the port shown for Silicon Labs CP210x USB to UART Bridge, and replace COM5 in the ingestion command. Do not assume COM10 is the ESP32: in the verified session, CIM identified COM5 as the CP210x board.

3. Start the Streamlit dashboard

Terminal 3 - local dashboard on port 8502

Port 8501 was already occupied by an older Docker dashboard on this computer. The working local dashboard was therefore started on port 8502. Local mode disables the dashboard password prompt for this local-only demonstration process.

Terminal 3 - Streamlit startup

```
cd "C:\Downloadss\aviation_safety_system_milestone13\aviation_safety_system_milestone13"
.\.venv\Scripts\Activate.ps1

$env:AVIATION_DEPLOYMENT_MODE = "local"
Remove-Item Env:AVIATION_APP_PASSWORD -ErrorAction SilentlyContinue

python -m streamlit run web_dashboard.py --server.address 127.0.0.1 --server.port 8502
```

Leave Terminal 3 open. Open the browser at: http://127.0.0.1:8502. Do not open port 8501 for this local instance.

Local access only
The dashboard is bound to 127.0.0.1 and local development mode is used for this machine. Do not use this password-free configuration for a shared or public deployment.

4. Select the live telemetry view

Dashboard control	Set to
Data source	API:SERIAL:ESP32
Scenario	All scenarios
Time range	Last 15 minutes
Auto-refresh	ON (5 seconds)
Action	Click Refresh data now

The sidebar data source must be API:SERIAL:ESP32. If it is set to All sources or a different source, select API:SERIAL:ESP32 for the live hardware view.

What confirms that live telemetry is flowing?

- The latest-sample message reports a recent age measured in seconds.
- The telemetry sample count increases as new packets arrive.
- The sensor module line reports DHT11, BMP280, MPU6050, and IR status.
- Risk score, system status, event count, and trend charts update from accepted telemetry.

5. Optional Ollama / AI demonstration

Live ESP32 telemetry does not require an LLM. Start or check Ollama only when the demonstration includes the AI Intelligence Lab.

Check local Ollama models and service

```
ollama list
Invoke-RestMethod http://127.0.0.1:11434/api/tags
```

The installed models from the benchmark session were gemma3:4b, qwen2.5:7b, and llama3.2:latest. If Ollama is not running, start the Ollama application/service. Use ollama serve in a separate terminal only when no Ollama service is already running.

The AI Intelligence Lab contains Live Diagnostic Explainer, Automated Benchmark Runner, Model Comparison Leaderboard, and Human Review & Audit Studio. Model inference can use CPU/GPU resources; avoid launching a second benchmark while one is running.

6. Troubleshooting quick reference

Symptom	Likely cause	Action
WinError 10013 / port 8080 busy	Older Docker API container is publishing port 8080, or another listener owns it.	Check docker ps. If aviation_safety_system_v10-ingest_api-1 is running, stop only that API container. Then start local python ingest_api.py.
HTTP 401 unauthorized	API and serial bridge are using different tokens.	Load AVIATION_INGEST_TOKEN from the same project .env in both PowerShell sessions. Never print or share the token.
HTTP 422: Only fused node_id=SYSTEM_AGGREGATE packets are accepted	The older Docker API expects the legacy aggregate-node contract.	Stop the old v10 API container and run the updated local Python API. Do not rename ESP32_ENV to SYSTEM_AGGREGATE.
Could not open port COM5	Wrong COM number, cable/driver issue, or Arduino Serial Monitor owns the port.	Close Serial Monitor. Reconnect USB. Identify Silicon Labs CP210x in Device Manager or Get-CimInstance Win32_SerialPort.
Port 8501 is not available	Older dashboard already uses 8501.	Run the local dashboard on port 8502.
Port 8502 is not available	A local Streamlit process may already be running.	Try opening http://127.0.0.1:8502 first. Do not launch a duplicate instance.
Dashboard asks for password	The process did not receive local-mode settings or password variable was set.	Restart Streamlit in a new terminal with AVIATION_DEPLOYMENT_MODE=local and remove Env:AVIATION_APP_PASSWORD for that process.
No fresh telemetry / stale sample age	Serial bridge stopped, API unavailable, source filter wrong, or packets rejected.	Check API /health; check Terminal 2 for HTTP 401/422; select API:SERIAL:ESP32 and click Refresh data now.
Initial malformed serial lines	ESP32 startup/reset output or blank/non-JSON lines.	A few initial skipped lines can occur. Persistent malformed lines mean verify JSON-only USB output at 115200 and correct firmware/port.

Never solve these errors by deleting the database, resetting Docker volumes, or changing the ESP32 node identity to satisfy the old API.

7. Safe shutdown when the demo is finished

Stop the services in this order. Press Ctrl+C in the terminal where each process is running; wait for it to return to the PowerShell prompt before closing that window.

Order	Terminal / process	Shutdown action
1	Terminal 2 - ESP32 serial ingestion	Press Ctrl+C first. This stops reading COM5 and releases the serial port.
2	Terminal 1 - local API Gateway	Press Ctrl+C to stop python ingest_api.py.
3	Terminal 3 - Streamlit dashboard	Press Ctrl+C to stop the local Streamlit server on port 8502.
4 (optional)	Ollama serve terminal, only if manually launched	Press Ctrl+C. If Ollama runs as a desktop/background service, it can be left running.

Do not do these things

- Do not run docker compose down -v. This can remove named volumes and persistent data.
- Do not delete data/aviation_safety.db or the .env file.
- Do not run the old Docker v10 API and local python ingest_api.py simultaneously on port 8080.
- Do not close Arduino Serial Monitor while Python is supposed to own COM5; close it before starting the bridge.
- Do not expose the local API or password-free local dashboard to the public internet.

Compact start checklist for the next demonstration

Check	Done
ESP32 powered; USB data cable connected; CP210x detected as COM5 (or current port).	[ ]
Arduino Serial Monitor closed.	[ ]
Old Docker v10 API container not running on port 8080.	[ ]
Terminal 1: same .env token loaded; python ingest_api.py running.	[ ]
API health check returns ok=True and database=ready.	[ ]
Terminal 2: same .env token loaded; serial ingestion on COM5 at 115200.	[ ]
Terminal 3: local Streamlit mode; dashboard running on port 8502.	[ ]
Browser: API:SERIAL:ESP32 selected; latest packet age is a few seconds.	[ ]
After demo: stop serial, API, and Streamlit with Ctrl+C.	[ ]

System boundary
ASRAM is an educational laboratory prototype. The deterministic Python risk engine is the authoritative source of prototype risk scores and states. Local LLM outputs are non-authoritative diagnostic hypotheses and must not control hardware or real aviation operations.

Runbook based on the verified Windows setup and successful live telemetry screenshot from 1 November 2026. Configuration values can change if Windows assigns another COM port or the project folder is moved.